

Stage 8 — Pricing, Billing, Quotas, and Feature Flags

Goal: Turn your multi-tenant, authenticated agent platform (Stages 1–7) into a real SaaS product with **plans, usage-based billing, per-plan quotas, and feature flags** — still keeping everything grounded in Postgres + external billing, no local hacks.

Stage 8 focuses on *business* architecture layered on top of your technical architecture.

1. Big Picture — What Stage 8 Adds

By now you have:[web:181][web:225][web:248]

- Multi-agent orchestrator and tools (Stages 1–2).
- Memory on Postgres + pgvector (Stage 3).[web:181]
- Guardrails & RBAC (Stage 4).[web:188][web:191]
- Evals & observability (Stage 5).[web:202][web:207]
- Production HTTP API & repo layout (Stage 6).[web:207]
- Auth, multi-tenancy, and rate limiting (Stage 7).[web:220][web:225][web:231]

Stage 8 adds four things:

1. **Plans & pricing model** — define free, pro, and enterprise tiers and what each gets.
2. **Usage metering & quotas** — measure usage per tenant (e.g., videos processed, tokens used) and enforce plan limits.[web:236][web:237][web:240][web:241][web:245]
3. **Billing integration** — send metered usage to Stripe (or similar) and charge customers automatically.[web:237][web:241][web:244][web:246][web:249]
4. **Feature flags** — roll out new features to subsets of tenants safely.[web:242][web:247]

This is where you go from "internal tool" to "billable SaaS API".

2. Choosing Your Pricing Metric (for LLM / Video Workflows)

Usage-based pricing best practices emphasize picking a metric that correlates with the actual value users get and that you can measure reliably.[web:236][web:237][web:240][web:245]

For your Creator Orchestrator, realistic options are:

- **Per video ingested** (e.g., each source video processed counts as 1 unit).
- **Per campaign** (a bundle of videos/posts per topic).
- **Per LLM token** (granular but less intuitive for creators).[web:241]

Given your earlier cost analysis, a simple and intuitive model is:

- **Primary metric:** VIDEO_INGESTED (1 per uploaded/linked source video).

- **Secondary metric (for internal control):** tokens consumed per tenant (already tracked in `runs.metadata).[web:241]`

You can then build **plans** around monthly video quotas and soft/hard caps.

Example plan sketch (for internal docs):

- **Free:** 5 videos/month, 1 seat, standard models.
- **Pro:** 50 videos/month, 5 seats, priority queue.
- **Enterprise:** custom quota, custom models, dedicated support.

3. Data Model for Plans, Subscriptions, and Quotas

Add a new migration `005_plans_billing_featureflags.sql`.

```
-- Billing plans
CREATE TABLE IF NOT EXISTS plans (
  id          UUID PRIMARY KEY,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  name        TEXT NOT NULL UNIQUE,          -- 'free', 'pro', 'enterprise'
  display_name TEXT NOT NULL,
  metadata    JSONB DEFAULT '{}'::jsonb      -- pricing details, feature flags, etc.
);

-- Tenant subscriptions to plans
CREATE TABLE IF NOT EXISTS subscriptions (
  id          UUID PRIMARY KEY,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  plan_id     UUID NOT NULL REFERENCES plans(id) ON DELETE RESTRICT,
  status      TEXT NOT NULL DEFAULT 'active', -- 'active', 'trialing', 'past_due'
  current_period_start TIMESTAMPTZ NOT NULL,
  current_period_end   TIMESTAMPTZ NOT NULL,
  external_ref TEXT,          -- e.g., Stripe subscription ID
  metadata    JSONB DEFAULT '{}'::jsonb
);

-- Usage counters per tenant & billing period
CREATE TABLE IF NOT EXISTS billing_usage (
  id          BIGSERIAL PRIMARY KEY,
  tenant_id   UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  period_start TIMESTAMPTZ NOT NULL,
  period_end   TIMESTAMPTZ NOT NULL,
  metric       TEXT NOT NULL,          -- 'VIDEO_INGESTED', 'TOKENS', etc.
  units        BIGINT NOT NULL DEFAULT 0,
  last_updated TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_billing_usage_tenant_metric
  ON billing_usage (tenant_id, metric, period_start, period_end);

-- Feature flags per tenant (simple built-in alternative to external services)
CREATE TABLE IF NOT EXISTS feature_flags (
```

```

    id          BIGSERIAL PRIMARY KEY,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    tenant_id    UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    flag_name     TEXT NOT NULL,
    enabled       BOOLEAN NOT NULL DEFAULT false,
    metadata      JSONB DEFAULT '{}'::jsonb,
    UNIQUE (tenant_id, flag_name)
);

```

This schema lets you:

- Define plans.
- Associate each tenant with a plan & billing period.
- Track usage for each metric per tenant per period.
- Toggle features per tenant.

Later, you will map `subscriptions.external_ref` to a Stripe subscription ID.

4. Seeding Plans and Assigning Tenants

Add to `scripts/seed.py` (conceptually):

```

import uuid
from datetime import datetime, timedelta, timezone
from memory.db import get_cursor

def seed_plans_and_subscriptions():
    now = datetime.now(timezone.utc)
    month_later = now + timedelta(days=30)

    with get_cursor() as cur:
        free_id = uuid.uuid4()
        pro_id = uuid.uuid4()

        cur.execute(
            """
            INSERT INTO plans (id, name, display_name, metadata)
            VALUES (%s, 'free', 'Free', %s),
                    (%s, 'pro', 'Pro', %s)
            ON CONFLICT (name) DO NOTHING
            """,
            (
                free_id, {"video_quota": 5},
                pro_id, {"video_quota": 50},
            ),
        )

    # Example: assign first tenant to free plan
    cur.execute("SELECT id FROM tenants ORDER BY created_at ASC LIMIT 1")
    row = cur.fetchone()
    if row:

```

```

tenant_id = row[0]
cur.execute(
    """
    INSERT INTO subscriptions (id, tenant_id, plan_id, current_period_start,
    VALUES (%s, %s, %s, %s, %s)
    ON CONFLICT (tenant_id) DO NOTHING
    """,
    (uuid.uuid4(), tenant_id, free_id, now, month_later),
)

```

This is not full billing; it simply sets up plans and attaches tenants to them. Stripe or another billing provider will later manage renewal periods and payment status.[web:237][web:244]

5. Usage Metering — Recording Billable Events

Usage-based billing requires accurate metering: your app must record every billable event tied to a tenant.[web:236][web:241][web:245][web:250]

For your product, a **billable event** is likely "video ingested".

5.1 Metering helper

billing/meter.py:

```

# billing/meter.py
from datetime import datetime, timezone
from typing import Literal

from memory.db import get_cursor

Metric = Literal["VIDEO_INGESTED", "TOKENS"]

def _current_billing_period() -> tuple[datetime, datetime]:
    # Simple monthly period, starting at calendar month boundaries
    now = datetime.now(timezone.utc)
    period_start = now.replace(day=1, hour=0, minute=0, second=0, microsecond=0)
    if period_start.month == 12:
        period_end = period_start.replace(year=period_start.year + 1, month=1)
    else:
        period_end = period_start.replace(month=period_start.month + 1)
    return period_start, period_end

def record_usage(tenant_id: str, metric: Metric, units: int) -> None:
    period_start, period_end = _current_billing_period()

    with get_cursor(tenant_id=tenant_id) as cur:
        cur.execute(
            """
            INSERT INTO billing_usage (tenant_id, period_start, period_end, metric, units)
            VALUES (%s, %s, %s, %s, %s)
            ON CONFLICT (tenant_id, metric, period_start, period_end)

```

```

DO UPDATE SET units = billing_usage.units + EXCLUDED.units,
              last_updated = now()
        "",
        (tenant_id, period_start, period_end, metric, units),
    )

```

You call `record_usage` from wherever ingestion happens (e.g., when a new video is registered for processing).

5.2 Hooking metering into your workflow

Wherever a "source video" enters the system (likely in a `/videos/upload` or similar endpoint, or at the top of a campaign in `/api/orchestrate`):

```

from billing.meter import record_usage

# inside your upload or orchestrate handler
record_usage(tenant_id, metric="VIDEO_INGESTED", units=1)

```

For LLM tokens, you can optionally add a call that increments `TOKENS` by the number of tokens consumed, using the metrics you already gather in Stage 5.[\[web:241\]](#)

6. Enforcing Plan Quotas (Hard/Soft Limits)

Quotas connect usage metering to **product behavior**: what plan allows how much usage. [\[web:236\]](#)[\[web:237\]](#)[\[web:240\]](#)[\[web:245\]](#)

6.1 Loading plan quotas

Add a helper to read video quota from the plan metadata.

`billing/plans.py`:

```

# billing/plans.py
from datetime import datetime, timezone
from typing import Optional

from memory.db import get_cursor

def get_active_subscription(tenant_id: str) -> Optional[dict]:
    now = datetime.now(timezone.utc)
    with get_cursor(tenant_id=tenant_id) as cur:
        cur.execute(
            """
            SELECT s.id, p.name, p.metadata, s.current_period_start, s.current_period_end
            FROM subscriptions s
            JOIN plans p ON s.plan_id = p.id
            WHERE s.tenant_id = %s
            AND s.status = 'active'
            AND s.current_period_start <= %s
            """
        )

```

```

        AND s.current_period_end > %s
    LIMIT 1
    """
    (tenant_id, now, now),
)
row = cur.fetchone()
if not row:
    return None
return {
    "subscription_id": row[0],
    "plan_name": row[1],
    "plan_metadata": row[2] or {},
    "period_start": row[3],
    "period_end": row[4],
}

def get_video_quota(plan_metadata: dict) -> int:
    return int(plan_metadata.get("video_quota", 0))

```

6.2 Checking quota before allowing ingestion

billing/quota.py:

```

# billing/quota.py
from fastapi import HTTPException, status
from billing.plans import get_active_subscription, get_video_quota
from billing.meter import _current_billing_period
from memory.db import get_cursor

def enforce_video_quota(tenant_id: str) -> None:
    sub = get_active_subscription(tenant_id)
    if not sub:
        raise HTTPException(status_code=status.HTTP_402_PAYMENT_REQUIRED,
                             detail="No active subscription")

    quota = get_video_quota(sub["plan_metadata"])
    if quota <= 0:
        return # unlimited

    period_start, period_end = _current_billing_period()

    with get_cursor(tenant_id=tenant_id) as cur:
        cur.execute(
            """
            SELECT COALESCE(SUM(units), 0)
            FROM billing_usage
            WHERE tenant_id = %s
               AND metric = 'VIDEO_INGESTED'
               AND period_start = %s
               AND period_end   = %s
            """
            (tenant_id, period_start, period_end),
        )

```

```

        used = cur.fetchone()[0]

    if used >= quota:
        raise HTTPException(
            status_code=status.HTTP_402_PAYMENT_REQUIRED,
            detail="Video quota exceeded for current billing period",
        )

```

Then, in your video ingest path or in `/api/orchestrate` before recording usage:

```

from billing.quota import enforce_video_quota

# before record_usage(...)
enforce_video_quota(tenant_id)
record_usage(tenant_id, "VIDEO_INGESTED", 1)

```

This gives you **plan-aware behavior**: free tenants are blocked after e.g. 5 videos/month; Pro tenants can go up to 50.[\[web:236\]](#)[\[web:237\]](#)[\[web:240\]](#)[\[web:245\]](#)

You can later add **soft limits** (warn first, then block) and email/webhook notifications when usage approaches quota.[\[web:237\]](#)[\[web:245\]](#)

7. Integrating with Stripe (or Another Billing Provider)

Rather than building invoices and payments yourself, plug into a mature billing platform — Stripe is common for usage-based SaaS and has emerging support for LLM token billing.[\[web:237\]](#)[\[web:241\]](#)[\[web:244\]](#)[\[web:246\]](#)[\[web:249\]](#)

7.1 Basic integration pattern

Common approach from guides and platform examples:[\[web:237\]](#)[\[web:244\]](#)[\[web:246\]](#)[\[web:249\]](#)

1. Stripe manages products, prices, and subscriptions.

- Each plan (free, pro, etc.) corresponds to a Stripe product + price.
- For usage-based metrics (videos, tokens), set up **metered prices** or use Stripe's Meter API or AI Gateway for token billing.[\[web:244\]](#)[\[web:246\]](#)

2. Your app emits usage events.

- Either directly to Stripe's Meter API when an event happens, or by periodically aggregating `billing_usage` and reporting.

3. Stripe generates invoices.

- At the end of each billing period, Stripe multiplies usage by per-unit price and charges the customer.[\[web:237\]](#)[\[web:244\]](#)[\[web:245\]](#)

4. Webhooks sync Stripe state back.

- When a subscription is created/updated/canceled, Stripe sends webhooks and you update your subscriptions table.

7.2 Reporting usage to Stripe

Two common patterns:[web:244][web:246][web:249]

- **Real-time metering:** call Stripe Meter API for each billable event.
- **Batch metering:** aggregate from `billing_usage` and send usage summaries periodically (hourly/daily).

Given you already aggregate `billing_usage` per tenant/period, batch metering is natural at first.

Pseudo-code for a scheduled script (e.g., `scripts/report_usage_to_stripe.py`):

```
# Pseudocode, not full implementation

from datetime import datetime, timedelta, timezone
from memory.db import get_cursor

def report_usage_to_stripe():
    # Find tenants and usage since last reporting window
    # For each tenant, send usage to Stripe via their Meter API or AI Gateway.
    # Store Stripe event IDs in billing_usage.metadata to avoid double-reporting.
    pass
```

For LLM tokens, you can either:

- Use Stripe's AI Gateway or Token Meter SDK to meter calls directly.[web:246]
- Or continue metering yourself and use Stripe's Meter API.

Full, production-grade Stripe integration is beyond this stage's scope, but your schema and usage tracking are aligned with how Stripe and similar platforms expect data.[web:237][web:241][web:244][web:246][web:249]

8. Feature Flags per Tenant

Feature flags let you roll out new models, tools, or capabilities to specific tenants without redeploying code.[web:242][web:247]

You already have a `feature_flags` table; now add a small helper layer.

8.1 Feature flag helpers

`feature_flags/service.py`:

```
# feature_flags/service.py
from typing import Optional
from memory.db import get_cursor

def is_feature_enabled(tenant_id: str, flag_name: str) -> bool:
    with get_cursor(tenant_id=tenant_id) as cur:
```



```

        cur.execute(
            """
            SELECT enabled
            FROM feature_flags
            WHERE tenant_id = %s AND flag_name = %s
            """,
            (tenant_id, flag_name),
        )
        row = cur.fetchone()
        return bool(row and row[0])

def set_feature_flag(tenant_id: str, flag_name: str, enabled: bool) -> None:
    with get_cursor(tenant_id=tenant_id) as cur:
        cur.execute(
            """
            INSERT INTO feature_flags (tenant_id, flag_name, enabled)
            VALUES (%s, %s, %s)
            ON CONFLICT (tenant_id, flag_name)
            DO UPDATE SET enabled = EXCLUDED.enabled
            """,
            (tenant_id, flag_name, enabled),
        )

```

8.2 Using flags in code

Examples:

- Switch models per tenant:

```

from feature_flags.service import is_feature_enabled

if is_feature_enabled(tenant_id, "use_premium_model"):
    writer_agent.model = "gpt-4.1" # hypothetical
else:
    writer_agent.model = "gpt-4o-mini"

```

- Gate new tools:

```

if is_feature_enabled(tenant_id, "auto_publish"):
    allowed_tools.append("schedule_post")

```

This follows the same principle as LaunchDarkly and similar feature flag platforms: your app reads a flag to decide behavior at runtime.[\[web:242\]](#)[\[web:247\]](#)

Later, you can swap your simple table for a dedicated feature-flag service if needed, but this built-in version is a good start.

9. Putting It All Together in the Request Flow

For a single `/api/orchestrate` request after Stage 8, the flow is:

1. Auth (Stage 7):

- `JWT` or `API key` → `user_id`, `tenant_id`.

2. Rate limit (Stage 7):

- `enforce_rate_limit(tenant_id, user_id)`.

3. Subscription & quota check (Stage 8):

- `get_active_subscription(tenant_id)`.
- `enforce_video_quota(tenant_id)`.

4. Feature flags (Stage 8):

- Configure agents/tools/models based on `feature_flags`.

5. Guardrails (Stage 4):

- Pre-LLM input guardrails.

6. Orchestrator & memory (Stages 2–3):

- Multi-agent workflow, Postgres+pgvector memory.

7. Guardrails (Stage 4):

- Post-LLM/content guardrails, tool RBAC.

8. Observability & evals (Stage 5):

- Metrics & evals recorded per `run_id`.

9. Usage metering (Stage 8):

- `record_usage(tenant_id, "VIDEO_INGESTED", 1)`.

This end-to-end path reflects modern patterns for commercial AI/LLM APIs: authenticated, multi-tenant, guarded, observable, and billable.[web:236][web:237][web:241][web:248]

10. Completion Checklist for Stage 8

You can call Stage 8 "done" when:

- [] Plans & subscriptions exist in DB, and each tenant has exactly one active plan.
- [] Billable events (e.g., videos ingested) are recorded in `billing_usage`.
- [] Quotas are enforced based on plan metadata; exceeding quota returns HTTP 402.
- [] Rate limiting (Stage 7) still applies; you can distinguish 429 (rate) vs 402 (quota/billing).
- [] Feature flags work; you can enable a new capability for one tenant and leave it off for others.
- [] You have a clear path to Stripe (or another billing provider) using your `billing_usage` data.

At that point, your Stage 1-8 architecture is a fully coherent, enterprise-style AI agent SaaS: multi-tenant, safe, observable, evaluable, and monetizable.